AI-POWERED UNIFIED SRI LANKAN TOURISM PLATFORM USING GRAPHRAG AND LLM ORCHESTRATION
INDIVIDUAL THESIS

KUMARASINGHE K.G.A.M.S.D.C.

IT22224866

Individual component: Smart AI Itinerary and Tour Plan Generator

Group ID: R26-IT-050

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specializing in Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

11 October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

I declare that this individual thesis describes my own research and development contribution to the Smart AI Itinerary and Tour Plan Generator of group R26-IT-050. Previously published work and relevant sources are acknowledged through IEEE numbered references. Distinct team contributions such as route-risk prediction, Neo4j hazard evidence, and vehicle recommendation are discussed only at integration boundaries.

The classification evaluation, software-test outcomes and demonstration observations are reported with their documented limitations. Application screenshots reproduced in this thesis were captured from the running synchronized application; they are software demonstration evidence, not an independent participant study.

Student signature: _____ Date: _____

SUPERVISOR CERTIFICATION

This thesis was prepared under the supervision of Ms. Dilani Lunugalage and co-supervision of Ms. Dulani Maheshika Jayasinghe.

Supervisor signature:  Date: __10/11/2026_____

Co-supervisor signature: _____ Date: _____

ABSTRACT

This thesis presents the Smart AI Itinerary and Tour Plan Generator, an individual component within an integrated Sri Lankan tourism platform. The component accepts a traveller starting location, interests, time budget and optional search radius, then constructs a feasible day itinerary from a bounded catalogue of verified Central Province points of interest. A Linear Support Vector Machine classifier estimates profile-POI relevance, after which a heuristic Genetic Algorithm selects feasible stops under a time budget. Haversine distance provides an estimated travel component and the selected stops are ordered for presentation. The system also supports individual-place replacement, feasible full-plan variation, stable target counts, itinerary-signature history and controlled exhaustion when no additional alternative exists.

KEYWORDS

Tourist itinerary planning; point of interest recommendation; Linear Support Vector Machine; Genetic Algorithm; Haversine distance; explainable recommendation; Sri Lanka tourism.

ACKNOWLEDGEMENTS

I thank my supervisor Ms. Dilani Lunugalage and co-supervisor Ms. Dulani Maheshika Jayasinghe for their guidance. I also acknowledge the members of group R26-IT-050 for the shared platform integration work and the Sri Lanka Institute of Information Technology for the academic environment in which this study was completed.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Problem statement 1
1.3 Research gap, aim and questions 1
1.4 Objectives and individual contributions 2
1.5 Scope, assumptions and thesis organisation 2
2 LITERATURE REVIEW AND RELATED WORK 3
2.1 Review method and scope 3
2.2 Tourist recommendation and trip design 3
2.3 Relevance classification and heldout evaluation 3
2.4 Optimisation, alternatives and repeat avoidance 4
2.5 Explainability and optional generative language 4
2.6 Distance calculations and limitations 4
3 RESEARCH METHODOLOGY 5
3.1 Research design 5
3.2 Study materials and catalogue 5
3.3 Input and processing procedure 5
3.4 Evaluation procedure and measures 6
3.5 Reproducibility and responsible AI 6
4 PRODUCT AND SOLUTION DESIGN 7
4.1 Architecture and module boundaries 7
4.2 Functional and non-functional requirements 8
4.3 Relevance and optimisation design 8
4.4 Time and travel design 8
4.5 Regeneration, signature control and explanation 8
5 IMPLEMENTATION, INTEGRATION AND TESTING 10
5.1 Technology stack and implementation 10
5.2 Frontend flow and presentation 10
5.3 Inter-component integration boundaries 10
5.4 Automated verification 11
5.5 Deployment and operational limits 11
6 RESULTS AND VALIDATION 12
6.1 Relevance model evaluation 12
6.2 Demonstration scenario and quantitative outputs 12
6.3 Original synchronized application screenshots 13
6.4 Summary of observed outcomes 22
7 DISCUSSION 23
7.1 Interpretation of research findings 23
7.2 Comparison with relevant techniques 23
7.3 Research and product contributions 23
7.4 Limitations and threats to validity 24
7.5 Implications 24
8 CONCLUSION AND FUTURE DIRECTIONS 25
8.1 Conclusion 25
8.2 Achievement of objectives 25
8.3 Individual contributions and future directions 25
REFERENCES 26
GLOSSARY 27
APPENDICES 28

LIST OF FIGURES

Figure 4.1. Documented Smart Itinerary processing flow and explanation boundaries 7
Figure 6.1. Completed traveller itinerary input form 14
Figure 6.2. Generated itinerary summary and ordered stops 15
Figure 6.3. Interactive map of ordered itinerary stops 16
Figure 6.4. Expanded place relevance and source-trace details 17
Figure 6.5. Deterministic route-selection explanation 18
Figure 6.6. Successful individual-place replacement 19
Figure 6.7. Successful feasible full-plan variation 20
Figure 6.8. Optional Gemini traveller guide 21
Figure 6.9. Relevance-gating and Genetic Algorithm selection stages 22

LIST OF TABLES

Table 1.1. Individual objectives and contributions 2
Table 3.1. Runtime POI catalogue distribution 5
Table 3.2. Input processing and output requirements 6
Table 4.1. Architecture responsibilities 7
Table 4.2. GA constraints and outputs 9
Table 5.1. Automated test results 11
Table 6.1. Model comparison 12
Table 6.2. Active v1 evaluation metrics 12
Table 6.3. v2 rejection evidence 12
Table 6.4. Demonstration results 13
Table 7.1. Limitations and future improvements 24
Table G.1. Glossary 27

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
GA	Genetic Algorithm
LLM	Large Language Model
POI	Point of Interest
SVM	Support Vector Machine
XAI	Explainable Artificial Intelligence

1 INTRODUCTION

1.1 Overall project context

The overall platform combines tourism planning with several team-owned services. This thesis isolates the Smart AI Itinerary and Tour Plan Generator. Its role is to make a bounded, preference-aware and time-constrained itinerary before optional handoff to the existing Safety Analyzer. Tourist trip design is commonly framed as selecting and ordering POIs under constraints such as preferences, distance and time [1].

The integrated tourism platform serves as the shared product context, but its services have different evidence bases and individual owners. This thesis examines the journey from a traveller-defined starting position and interest profile to a bounded sequence of points of interest (POIs). The output is an itinerary: an ordered list of recommended places with planned visit time and estimated inter-stop travel, not a road-safety assurance or an accommodation quotation.

The component is designed around a practical day-planning question: which relevant places can be visited within the time available, using a modest and clearly stated travel approximation? Its relevance stage and heuristic optimisation stage solve different subtasks. The classifier supports candidate suitability; the optimiser creates a feasible set and ordering.

1.2 Problem statement

A traveller may know available time and broad interests but still face a large decision space. A useful prototype must transform these inputs into a feasible plan without overstating precision. The component therefore operates within a verified Central Province catalogue and reports estimated rather than live-road travel.

Selecting attractions solely by popularity does not address the traveller's time budget, geographic starting location or interests. Conversely, choosing the geographically nearest places may waste the trip on points of little interest. The research problem is therefore constrained rather than unconstrained: generate a defensible itinerary using the records and evidence available, while communicating limitations in travel estimation and relevance prediction.

The prototype deliberately limits the searchable catalogue to source-traced Central Province POIs. This makes the demonstration reproducible and avoids implying comprehensive nationwide coverage.

1.3 Research gap, aim and questions

Within the integrated platform, a research and product gap was the absence of a bounded, preference-aware itinerary workflow that could generate an initial feasible plan, preserve the accepted stops during a targeted replacement, and avoid immediately recycling recently displayed plan variants. The contribution is not a newly discovered optimisation theorem or proof of global optimality; it is a functioning integration of evidence-limited relevance and deterministic feasibility logic.

Aim: develop and evaluate a source-traced, interest-aware and time-budget-constrained itinerary generator with explainable results and controlled regeneration.

RQ1: How accurately does the active relevance model classify human-reviewed traveller-profile/POI examples within the documented heldout evaluation?

RQ2: Can the implementation construct and update feasible ordered itineraries while retaining defined time-budget and stop-count constraints?

RQ3: How are relevance, travel estimates, regeneration decisions and optional language-model explanations presented without overstating certainty?

1.4 Objectives and individual contributions

The aim was to design and validate a constrained itinerary generator that provides reproducible route alternatives. Objectives were to validate traveller input, estimate relevance, filter candidates, select a feasible ordered route, provide controlled regeneration, explain the deterministic route choice and integrate the result with the wider platform.

Contribution type	Individual component contribution
Research	Evaluation of a v1 Linear SVM relevance gate on a stated human-heldout set
Product	Itinerary input, optimization, replacement, variation, map presentation and explanation
Integration	Authenticated navigation, optional persistence and ordered-stop handoff to Safety Analyzer
Boundary	Safety/risk, Neo4j, weather and vehicle decisions remain team components

Table 1.1. Individual objectives and contributions

The individual technical boundary includes input validation, relevance gating, candidate selection, estimated travel, ordering, replacement, variation, repeat avoidance and itinerary explanation. Authentication-aware navigation and persistence are shared integration features required to demonstrate the normal user experience. The Safety Analyzer can receive the final ordered stops; its predictive model and recommendations are not evaluated as personal research results here.

1.5 Scope, assumptions and thesis organisation

The current working geographic scope is Central Province, represented by 40 source-traced locations. The active classifier was evaluated on a smaller, earlier Kandy-based set of 20 POIs. The application does not verify opening hours, live traffic, road distance, parking, walking legs or guaranteed return travel. It computes estimates with Haversine distance, assumed speed and a traffic buffer.

Chapter 2 places the component within relevant itinerary-design, classification and explanation literature. Chapter 3 defines the datasets and evidence. Chapters 4 and 5 document solution design and integration. Chapter 6 presents verified metrics and original application screenshots. Chapters 7 and 8 discuss interpretation, limitations and future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

This is a focused narrative review of literature directly relevant to the implemented solution; it is not a systematic search claiming exhaustive database coverage. Sources address the tourist-trip-design problem, mobile tourism recommendation, support-vector classification, optimisation, explanation, geographical distance and language models. The seven references are retained from the verified thesis bibliography. The review motivates design choices but is not presented as evidence that this particular implementation is superior to commercial systems.

2.2 Tourist recommendation and trip design

Tourist recommendation is context-sensitive because preferences, location, time and device interaction affect useful suggestions [2]. Ruiz-Meza and Montoya-Torres describe the tourist trip design problem as an operations-research problem with POI and route constraints [1]. These sources motivate the component architecture but do not validate the project-specific results.

Tourist trip design can be understood as a constrained selection and ordering problem. Each point of interest carries attraction value, a possible visit-duration estimate and a geographical position. A plan is practical only if the total estimated visit and travel time remains inside the traveller's budget. This differs from ranking isolated attractions because the usefulness of one stop depends on the other stops already chosen and the resulting route.

The component therefore separates candidate relevance from route feasibility. The literature establishes why both preference and constraint handling matter, while the observed effectiveness of the implemented version remains tied to project-specific tests and demonstrations [1], [2].

2.3 Relevance classification and heldout evaluation

Support-vector networks provide a well-established approach to classification by constructing separating decision boundaries in a feature space [3]. In this project, the classifier predicts profile-POI relevance rather than the quality of a complete itinerary. It is consequently used to gate candidates before route optimization.

Support-vector learning motivates the Linear SVM used for profile–POI relevance classification [3]. The input to this decision is a traveller profile paired with a candidate location, not a whole itinerary. An item may be classified as relevant without being chosen: the final plan still needs to satisfy the time budget and optimisation criteria.

The recorded 60-example human-heldout assessment should be interpreted with its class imbalance and restricted location scope. A high nominal accuracy alone would not establish suitability for every preference group, so balanced accuracy and macro F1 are reported alongside overall accuracy.

2.4 Optimisation, alternatives and repeat avoidance

The heuristic Genetic Algorithm (GA) searches feasible candidate combinations under a constrained time budget. Heuristic search can balance competing objectives but does not guarantee a globally optimal route.

In this study the final programme applies feasibility checks and deterministic ordering and supports controlled alternative generation.

Replacement and variation are distinct behaviours. Replacement attempts to change one selected POI while preserving accepted stops. Full-plan variation seeks a different feasible signature while maintaining a stable number of stops where possible. A recent-plan signature history prevents trivial cycles; a controlled exhaustion state is preferred when no unseen acceptable alternative remains.

2.5 Explainability and optional generative language

Genetic methods are applicable when itinerary objectives combine traveller preferences and travel efficiency [7]. Explanation is separately important because a recommendation should communicate why an outcome was selected [4]. The implementation generates deterministic route-selection reasons from the finalized plan. An optional LLM guide converts this fixed evidence into traveller-friendly wording; literature recognizes LLMs can occupy different recommender-system stages [6], but this design deliberately prevents it from ranking or optimizing.

Explanation is both a product feature and an evidence boundary. The deterministic explanation describes final route properties, such as stop count, selected interests, time utilisation and travel assumptions. The optional Gemini guide rephrases already selected and ordered stops; it is not allowed to introduce a different recommended stop, recalculate scores or re-optimise the plan. This separation addresses the distinction between an explanation surface and a decision-maker [4], [6].

2.6 Distance calculations and limitations

The route uses Haversine-style geographic distance as an estimate. It is not live traffic, road-network routing, parking time or walking-time modelling. Geodesic research demonstrates the distinction between geometric calculations and the richer accuracy requirements of real-world navigation [5].

Haversine-style straight-line distance is computationally simple and auditable; it is not a road-network path. Even accurate geographic coordinates cannot, by themselves, account for hills, traffic, one-way roads or accessible walking routes. Accordingly, this component reports estimated rather than operationally guaranteed travel durations. Geodesic method literature supplies relevant background without validating the prototype's road-time approximation [5].

3 RESEARCH METHODOLOGY

3.1 Research design

The work used an applied design-and-evaluate approach. Repository tests, a bounded human-heldout relevance evaluation and a reproducible API demonstration supplied the evidence. No formal stakeholder or user study was completed.

The methodology has three evidence streams. First, a labelled profile–POI evaluation assesses the active relevance classifier. Second, automated tests verify documented functions and integration contracts. Third, a manually exercised synchronized frontend demonstrates visible itinerary generation, source details, replacement, variation, the optional guide and selection-stage presentation. The last stream establishes a software behaviour sequence, not independent field effectiveness.

No new training experiment or participant study was carried out to create this thesis revision. Existing numerical results remain traceable to the documented heldout set and recorded automated tests.

3.2 Study materials and catalogue

The runtime catalogue contains 40 verified, source-traced Central Province POIs. It comprises 20 Kandy POIs, 10 Matale POIs and 10 Nuwara Eliya POIs. The relevance dataset has 800 rows, each a profile-POI training or evaluation pair; it is not a catalogue of 800 tourist places.

District	Runtime POIs	Role
Kandy	20	Largest prototype cluster
Matale	10	Central Province coverage
Nuwara Eliya	10	Central Province coverage
Total	40	Runtime verified catalogue

Table 3.1. Runtime POI catalogue distribution

The 800 rows used for relevance training/evaluation are profile–POI examples, not 800 tourist attractions. The application runtime catalogue is separate and contains 40 POIs: 20 in Kandy, 10 in Matale and 10 in Nuwara Eliya. The locations are source-traced in the prototype catalogue. This distinction is essential for interpreting both system coverage and the scale of the classification task.

The active v1 evaluation was completed using an earlier bounded 20-POI Kandy set. Its heldout metrics cannot be transferred automatically to all 40 currently loaded locations. Any claim of improved geographic generalisation requires new independently documented evaluation.

3.3 Input and processing procedure

Stage	Input or processing	Output
Input	Latitude, longitude, interests, hours, minutes, radius	Validated traveller request
Relevance	Profile-POI features and Linear SVM v1	Candidate relevance gate
Optimization	Feasible candidates and time budget	Selected ordered stops

Stage	Input or processing	Output
Explanation	Final route evidence	Deterministic explanation and optional guide
Integration	Ordered stop list	Safety Analyzer handoff

Table 3.2. Input processing and output requirements

A generation request supplies starting latitude and longitude, interest categories, hours, minutes and a search radius. Validation is followed by bounded catalogue retrieval and relevance estimation. The optimisation procedure searches for feasible combinations and the result returns ordered stop information plus travel/visit summaries. The calculation explicitly excludes a claim of live routing.

The frontend allows the traveller to inspect relevance details for individual selected stops, request a targeted replacement, generate a full-plan variation and open a route map. Accepted alternatives must pass the same feasibility rules rather than merely changing visible text.

3.4 Evaluation procedure and measures

The active v1 model was evaluated on an earlier bounded 20-POI Kandy human-heldout set. It was not independently evaluated across the current 40-POI runtime catalogue. The heldout metrics measure profile-POI relevance, not end-to-end itinerary quality. A v2 candidate was compared and rejected before runtime activation.

The classification evaluation reports correct predictions, accuracy, balanced accuracy, macro F1 and weighted F1 on the stated human-heldout cases. Accuracy is the share of predictions that match the human labels. Balanced accuracy averages class-level recall. Macro F1 weights each class equally, while weighted F1 accounts for class frequencies.

The comparison includes a dummy baseline, Decision Tree, Random Forest and Linear SVM on the same 60 heldout examples. A proposed v2 candidate did not meet activation criteria and is explicitly excluded from runtime claims. Tests of software behaviour are reported separately from model statistics; successful endpoint tests do not constitute independent evidence that travellers prefer the resulting routes.

3.5 Reproducibility and responsible AI

The implementation protects the deterministic route path from optional external language-model output. MongoDB persistence is optional for itinerary generation but is required by the normal authentication-aware UI flow. Tests use controlled Gemini error paths and do not require a real API key.

The synchronized repository revision used for the application evidence is sync/main-origin-integration-20261011 at merge commit ad691e42, as recorded in project notes. The 40-POI catalogue and active model are preserved. The demonstration uses a named time budget and fixed input values, which permits direct comparison of displayed results with previously verified API observations.

The reported full Python suite has four errors attributable to unavailable reviewer-workbook fixtures. The absent workbooks were not fabricated to manufacture a passing result. Environment secrets are omitted from this report. MongoDB connectivity was required for the authenticated application demonstration, while raw itinerary generation can be exercised without persistence.

4 PRODUCT AND SOLUTION DESIGN

4.1 Architecture and module boundaries

The React Native and Expo frontend validates and submits itinerary inputs. The Node and Express backend validates the request, forwards it to the Flask service, maps controlled failures and optionally persists generated itineraries for authenticated users. The Python and Flask service loads the verified catalogue, performs SVM inference, builds feasible alternatives, orders stops and returns structured results.

Layer	Responsibilities
Frontend	Authentication-aware navigation, form, cards, map, regeneration controls and explanation presentation
Node Express	Itinerary API, validation, timeout/error forwarding and optional persistence
Python Flask	Catalogue loading, SVM inference, candidate filtering, optimization, ordering, XAI and controlled exhaustion
Team integration	Hotel/vehicle/location flow, user session, MongoDB and Safety Analyzer handoff

Table 4.1. Architecture responsibilities

The system can be read as a sequence of responsibility boundaries: (1) React Native/Expo obtains and validates traveller input, (2) Express accepts the API request and communicates with the AI service, (3) Python/Flask loads catalogue and relevance resources, performs relevance gating and heuristic itinerary selection, and (4) the frontend presents route evidence with optional downstream team integration.

The Node service handles bounded communication, controlled errors and optional user-associated persistence. Source-specific services remain separated so failure of an optional explanation does not alter the base itinerary. This modular organisation also prevents the Safety Analyzer's road-risk output from being misrepresented as an itinerary classifier result.

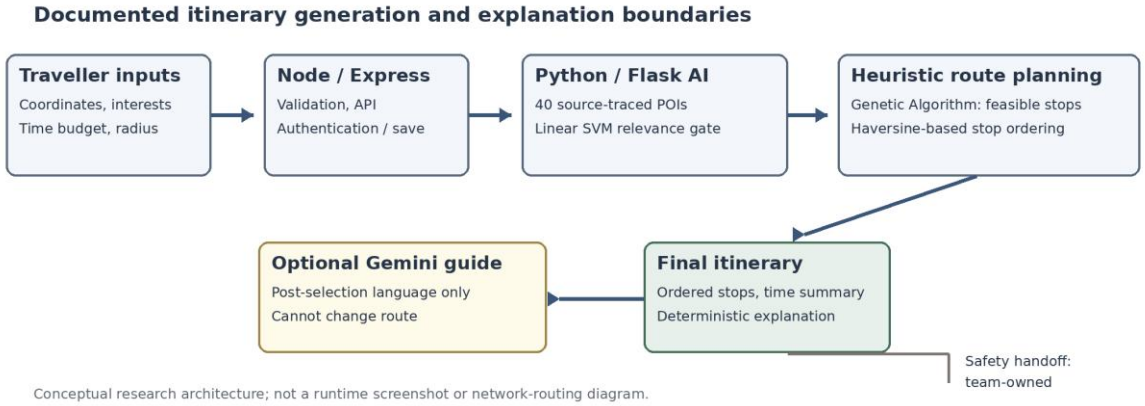


Figure 4.1. Documented Smart Itinerary processing flow and explanation boundaries.

Figure 4.1 separates deterministic itinerary construction from the optional language guide. The final ordered stops may be forwarded to a team-owned safety service, but its safety prediction and vehicle decision logic are outside this individual research contribution.

4.2 Functional and non-functional requirements

Functional requirements include: valid coordinates and preferences; a bounded catalogue; relevance-based candidate gating; time-feasible stop selection; ordered output; detailed place evidence; one-place replacement; full-plan variation; readable route rationale; and optional traveller-facing guide.

Non-functional requirements include predictable behaviour under the documented scenario, controlled failure responses, authentication-aware access, a fallback when Gemini is unavailable and preservation of model/dataset boundaries. Runtime throughput, real-road arrival accuracy and nationwide generalisability were not independently benchmarked.

4.3 Relevance and optimisation design

Input validation rejects malformed coordinates, invalid time values and unsupported starting locations. The classifier assigns relevance evidence to profile-POI pairs. Candidate gating reduces the optimization search space. The GA uses a route representation of selected POIs, an initial population of feasible candidate combinations, fitness considerations including preference relevance and feasible time use, selection, crossover and mutation. Exact parameter values are intentionally not stated where they are not documented as stable research evidence.

The GA is a heuristic selector rather than a classifier. Candidate itineraries represent combinations of filtered POIs. The search uses feasibility constraints with relevance and practical use of the available time as considerations. Selected stops are then ordered using estimated geographic legs. Exact hyperparameter values are not claimed here because stable archival evidence for every setting was not included in the draft.

The interface may report an explicit relevance gate or content fallback, a fixed proximity prior used for coarse ranking, deterministic seeded heuristic selection and exact minimum-Haversine ordering over the chosen stop set. These are application-displayed stages, not proof that all conceivable itineraries were enumerated. The user-facing explanation explicitly states that no global optimum is claimed.

4.4 Time and travel design

The estimated used time is the sum of planned POI visit durations and estimated inter-stop travel. Remaining time is the original budget less total used time. Utilisation is total estimated used time divided by the available budget. For the recorded five-hour demonstration, 273 of 300 minutes yields 91.0% utilisation.

Straight-line geographic distance uses the Haversine relationship. The application explanation names an assumed 30 km/h travel speed and a 1.25 traffic buffer for the displayed demonstration. These assumptions can be useful for a prototype but do not validate actual driving conditions, parking requirements or return transport.

4.5 Regeneration, signature control and explanation

Replacement excludes the requested POI while keeping accepted stops where feasible. Full-plan variation excludes the current plan and checks canonical POI-ID signatures so recently displayed plans are not repeated. The target number of stops remains stable unless feasibility requires a controlled response. If no

acceptable alternative remains, the API returns controlled exhaustion rather than fabricating a plan. Deterministic explanations derive from final route evidence. Gemini receives only finalized information and may provide a guide; it cannot alter selection, ranking or optimization.

Constraint or output	Design treatment
Time budget	Feasibility check before response
Travel	Haversine estimate; no road or traffic claim
Stop count	Stable target where feasible
Replacement	Exclude selected POI and preserve accepted stops where feasible
Variation	Exclude recent itinerary signatures
No alternative	Controlled exhaustion response

Table 4.2. GA constraints and outputs

The replacement feature changes the requested POI while preserving accepted stops when the remaining constraints permit. The full-plan variation feature seeks a distinct feasible set, rejecting recently emitted canonical signatures. This is important because a naïve alternative generator could repeatedly cycle between two superficially different screens without producing useful choices.

The language-model layer receives the already finalised stop sequence and may explain each stop in route order. When its output is missing, malformed or unavailable, the deterministic plan and rationale remain usable. A successful Gemini explanation in the screenshot evidence should be interpreted as descriptive output, not proof of independent factual verification of every attraction narrative.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Technology stack and implementation

The synchronized branch integrates local itinerary authentication, user persistence and Gemini configuration with remote hotel pricing, vehicle flow, coordinate routing, route-map location picking, current-location behaviour and mobile-back navigation. Smart Itinerary remains reachable from shared navigation. Persistence failures do not prevent direct itinerary generation, although normal frontend sign-in depends on MongoDB.

The web demonstration is served by the Expo/React Native frontend. The Express backend implements itinerary API validation and forwards relevant requests to the Python Flask AI service. The service handles catalogue loading, active Linear SVM inference, candidate evaluation, itinerary optimisation, ordering and explanation composition. MongoDB supports authenticated user accounts and itinerary persistence.

The observed local demonstration required correctly directed service connections and a live MongoDB instance. Those operational dependencies are relevant to reproducing the screenshots; they do not change the classifier or constitute a new research performance experiment.

5.2 Frontend flow and presentation

The traveller-facing flow begins with a starting-location map or coordinates, multiple interest categories, a time budget and travel radius. After generation, the interface presents a summary of visit minutes, travel minutes, total time, remaining time and utilisation. Each ordered stop supports a research-detail expansion and one-place replacement.

The results view also offers a map, generation of an alternative feasible plan, a deterministic explanation and an optional Gemini guide. An itinerary can be handed off to a shared vehicle/safety component, but this thesis does not evaluate that downstream component's risk prediction or vehicle results.

5.3 Inter-component integration boundaries

After the route is finalized, the ordered stops may be passed to the Safety Analyzer. That handoff is integration evidence only. Neo4j hazards, weather evidence, road-risk analysis and vehicle recommendation are not individual results of this component.

Integration boundaries are identified explicitly. The itinerary generator owns the ordered output and relevant selection rationale. Authentication and persisted itinerary retrieval are shared platform services. The Safety Analyzer, road-risk prediction, Neo4j hazard evaluation, weather evidence and vehicle selection are separate research components. Handoff is an API/data-flow integration event, not an assertion of model ownership.

5.4 Automated verification

Test area	Result	Interpretation
Backend Node tests	65 passed, 0 failed	API, persistence, safety integration and hotel/vehicle contracts passed

Test area	Result	Interpretation
Frontend tests	18 passed, 0 failed	Vehicle mapping and itinerary regeneration controls passed
Frontend lint	Passed	Static lint validation passed
Expo web export	Completed successfully	Web bundle verification completed
Focused Python tests	72 passed, 0 failed, 0 errors	Runtime itinerary and model behaviour passed
Full Python suite	122 executed; 4 errors	Missing separately supplied reviewer workbooks

Table 5.1. Automated test results

The recorded backend suite passed 65 tests with no failures. Frontend itinerary tests passed 18 with no failures; linting passed and Expo web export completed. Focused Python itinerary/model tests passed 72 with zero failures and zero errors. The complete Python suite executed 122 tests and reported four errors associated with missing separately supplied reviewer-workbook fixtures. It is therefore inaccurate to call the full Python suite passed.

The focused test coverage addresses generation, relevance prediction, replacement, variation, stop target handling, recent-signature cycles, exhaustion responses, Gemini parsing/fallback, deterministic explanations and time-budget checks. This is verification of code behaviours, not external validation of final tourist outcomes.

5.5 Deployment and operational limits

The authenticated web prototype was demonstrated with a running Expo interface, Express backend, Flask AI service and local MongoDB. These observations concern local operation only; they do not establish a deployed public production service or a measured uptime guarantee. A failed downstream safety graph connection need not invalidate a successfully generated itinerary.

Sensitive environment values and credentials are intentionally excluded. Screenshots were taken from the real application after normal sign-in, not a fabricated session or mocked result.

6 RESULTS AND VALIDATION

6.1 Relevance model evaluation

Linear SVM v1 achieved 41 correct predictions from 60 human-heldout cases, or 68.33% accuracy. Balanced accuracy was approximately 56.52%, macro F1 approximately 51.96%, and weighted F1 approximately 62.82%. The difference between overall and balanced metrics indicates that minority-class recall should not be hidden. The evidence supports continued use as a bounded relevance gate, not a claim of high-quality complete itineraries.

Model	Heldout accuracy	Decision
Dummy baseline	56.67%	Baseline
Decision Tree	66.67%	Below v1
Random Forest	66.67%	Below v1
Linear SVM v1	68.33%	Active runtime classifier

Table 6.1. Model comparison

Metric	v1 result	Interpretation
Correct predictions	41/60	Human-heldout profile-POI pairs
Accuracy	68.33%	Overall correctness
Balanced accuracy	56.52%	Class-sensitive performance is moderate
Macro F1	51.96%	Weak minority-class performance remains
Weighted F1	62.82%	Class-frequency-weighted result

Table 6.2. Active v1 evaluation metrics

Candidate	Correct predictions	Accuracy	Decision
v2 candidate	36/60	60.00%	Rejected; not runtime activated

Table 6.3. v2 rejection evidence

The active Linear SVM v1 had 41 correct predictions out of 60 human-heldout profile–POI pairs, achieving 68.33% accuracy. The same evaluation reports approximately 56.52% balanced accuracy, 51.96% macro F1 and 62.82% weighted F1. The gap between raw accuracy and class-balanced measures cautions against overstating performance on less frequent relevance labels.

The dummy baseline achieved 56.67% and both Decision Tree and Random Forest achieved 66.67% on the stated comparison. The observed SVM advantage is small and has not been established as statistically significant. The v2 candidate obtained 36/60 correct (60.00%) and was rejected; it is not the active runtime model. No classifier accuracy is assigned to the Genetic Algorithm, and no claim of 94% applies to this evaluated relevance component.

6.2 Demonstration scenario and quantitative outputs

A verified API demonstration used latitude 7.290600, longitude 80.633700, interests History, Culture and Nature, five hours, and a 30 km radius. The initial route listed Giragama Walawwa, Temple of the Sacred Tooth Relic, National Museum of Kandy and British Garrison Cemetery. It used 270 visit minutes and 3

estimated travel minutes, for 273 total minutes, 27 remaining minutes and 91.0% utilization. Replacing British Garrison Cemetery returned Giragama Walawwa, Kandy Lake, Temple of the Sacred Tooth Relic and National Museum of Kandy. This used 285 visit minutes and 3 travel minutes, for 288 total minutes, 12 remaining minutes and 96.0% utilization. These are pre-synchronization API demonstration results and must be revalidated in the synchronized UI before final submission.

Scenario	Stops	Visit	Travel estimate	Total	Remaining	Utilization
Initial	Giragama Walawwa; Temple; Museum; Cemetery	270 min	3 min	273 min	27 min	91.0%
Replacement	Giragama Walawwa; Kandy Lake; Temple; Museum	285 min	3 min	288 min	12 min	96.0%

Table 6.4. Demonstration results

The synchronized application showed the same initial four stops as the previously recorded API demonstration: Giragama Walawwa, Temple of the Sacred Tooth Relic, National Museum of Kandy and British Garrison Cemetery. The frontend displayed 270 planned visit minutes, 3 estimated travel minutes, 273 total minutes, 27 minutes remaining and 91.0% utilisation. It also displayed that the plan was saved in the prototype database.

A targeted replacement of British Garrison Cemetery produced Giragama Walawwa, Kandy Lake, Temple of the Sacred Tooth Relic and National Museum of Kandy. The application displayed 285 visit minutes, 3 estimated travel minutes, 288 total minutes, 12 minutes remaining and 96.0% utilisation. This is evidence of a working replacement under the demonstrated inputs, not a statistically representative improvement estimate.

A separate successful full-plan variation displayed Giragama Walawwa, Temple of the Sacred Tooth Relic, National Museum of Kandy and Arthur's Seat, using 270 visit minutes and 4 estimated travel minutes, for 274 total minutes and 26 minutes remaining (91.3%). This additional variation was observed directly in the supplied genuine frontend screenshot.

6.3 Original synchronized application screenshots

Figures 6.1–6.9 reproduce screenshots supplied from the actual authenticated CeylonGo interface. They are retained as captured without replacement by fabricated UI. Because a long browser screenshot can make text small in print, each image is followed by a short interpretation of what it visibly verifies.

6.3.1 Completed traveller itinerary input form

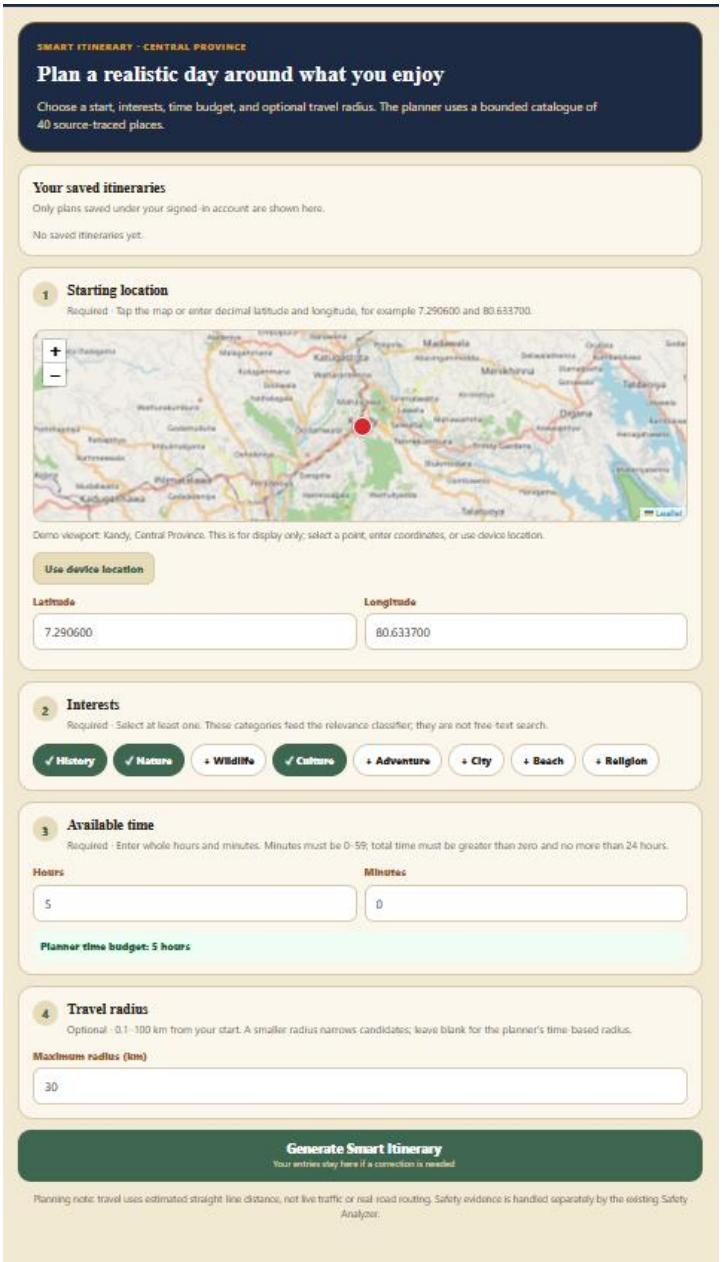


Figure 6.1. Completed traveller itinerary input form.

The form records the demonstration starting coordinates (7.290600, 80.633700), History, Nature and Culture, five available hours and a 30-kilometre search radius.

6.3.2 Generated itinerary summary and ordered stops

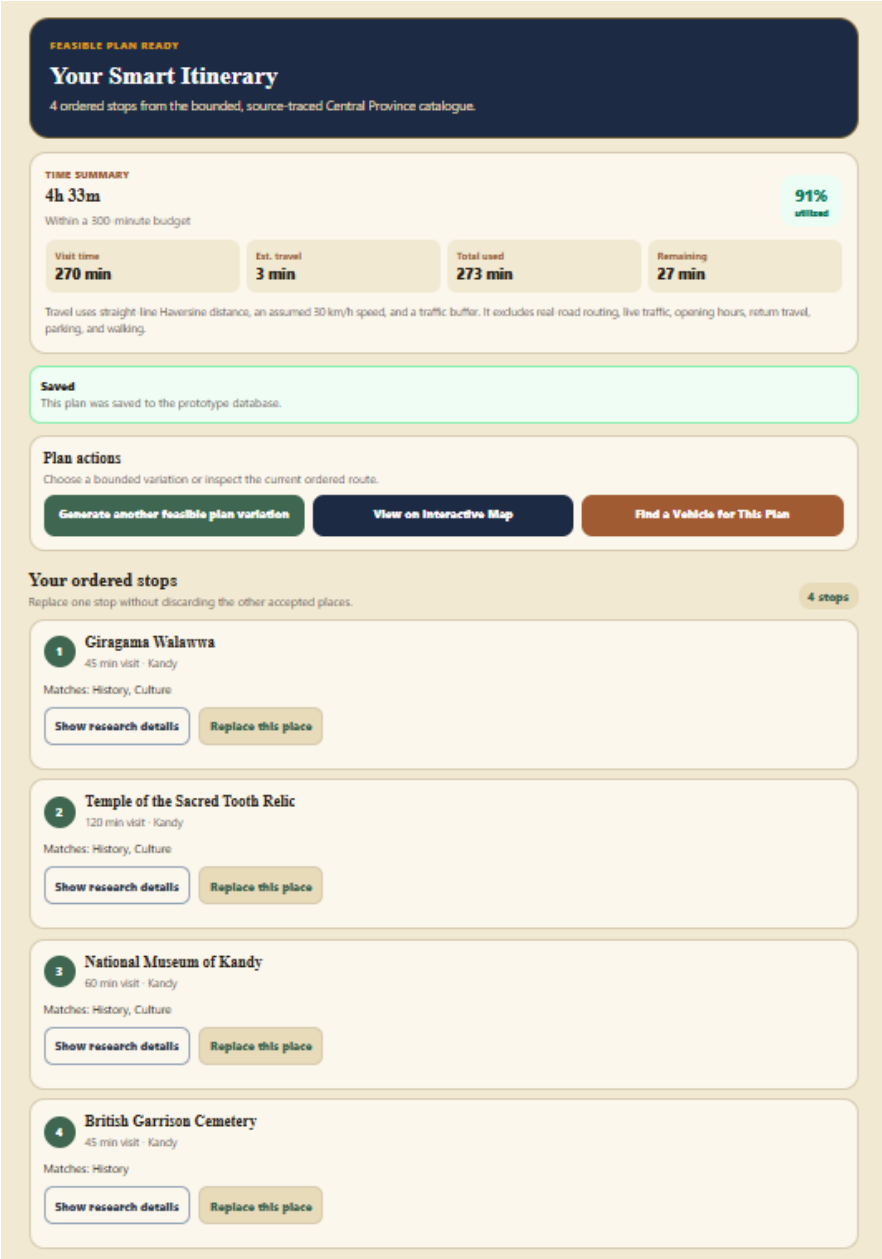


Figure 6.2. Generated itinerary summary and ordered stops.

The authenticated results view shows four ordered locations, 273 minutes used, 27 minutes remaining, 91.0% utilisation and a visible saved status.

6.3.3 Interactive map of ordered itinerary stops

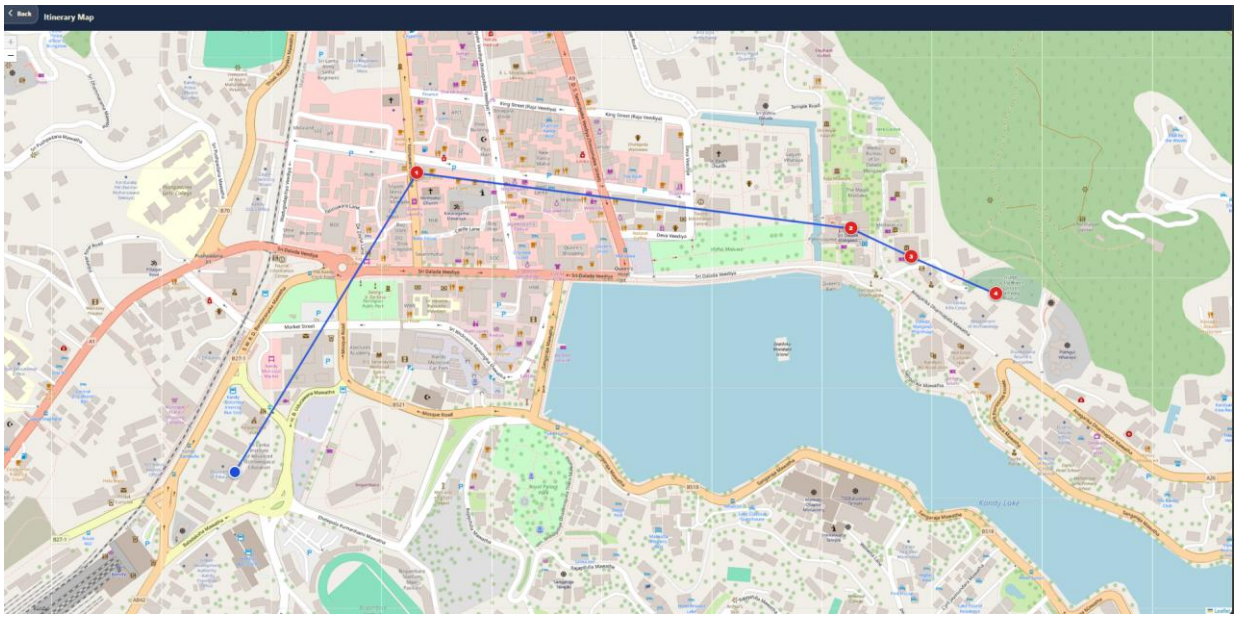


Figure 6.3. Interactive map of ordered itinerary stops.

The interactive Leaflet map displays numbered points and connecting straight-line legs around Kandy. The lines are not asserted to be real-road paths.

6.3.4 Expanded place relevance and source-trace details



Figure 6.4. Expanded place relevance and source-trace details.

Expanded research details show the selected stop's relevance class, classification score, combined score, estimated incoming leg and source status. The classification score is not a satisfaction probability.

6.3.5 Deterministic route-selection explanation

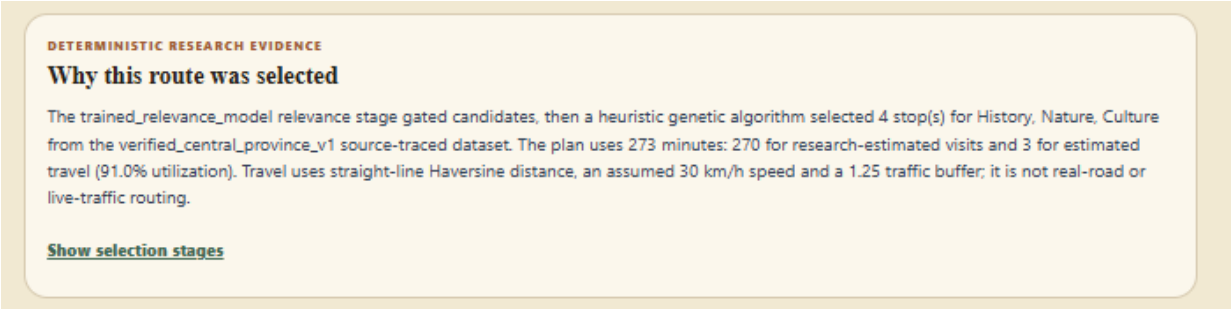


Figure 6.5. Deterministic route-selection explanation.

The deterministic explanation identifies relevance gating, heuristic selection, four stops, a five-hour budget and stated Haversine travel assumptions.

6.3.6 Successful individual-place replacement

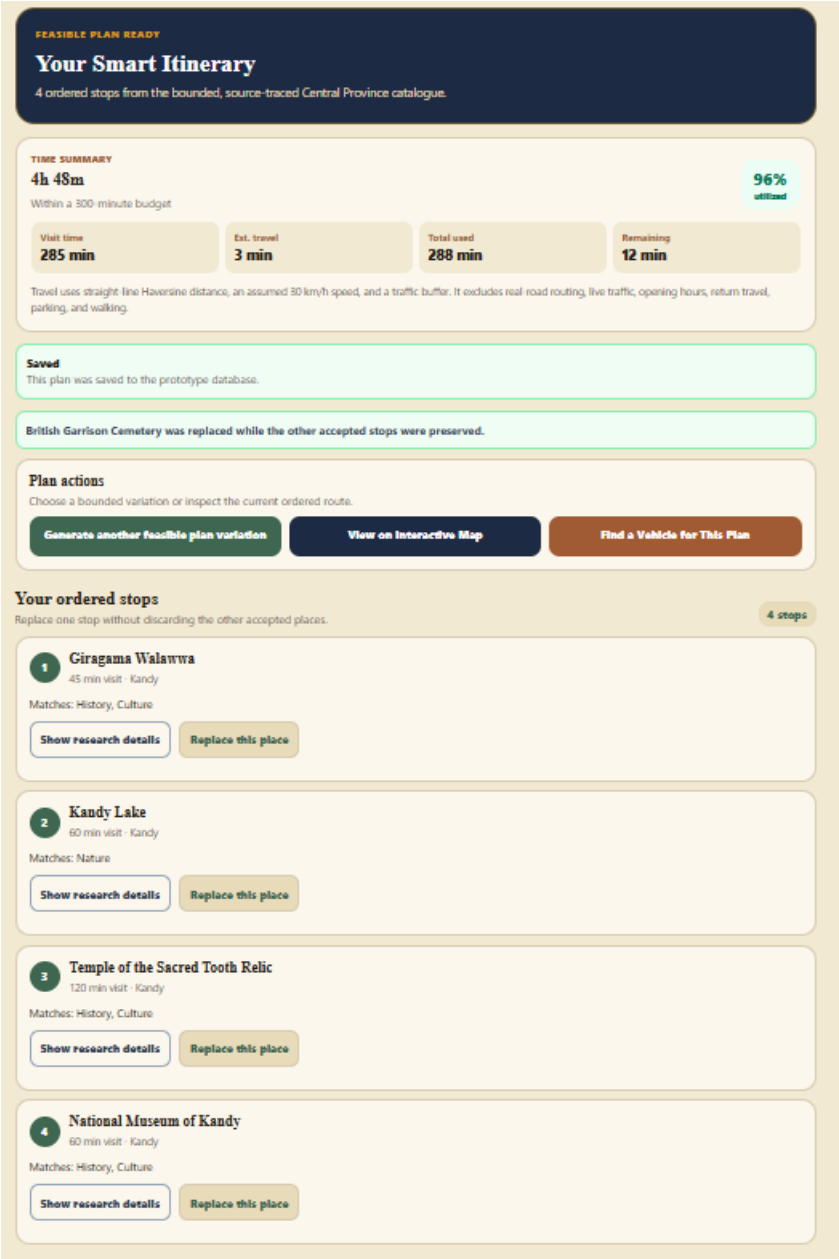


Figure 6.6. Successful individual-place replacement.

The replacement view confirms British Garrison Cemetery was replaced while accepted stops remained, yielding Kandy Lake and a 96.0% time utilisation summary.

6.3.7 Successful feasible full-plan variation

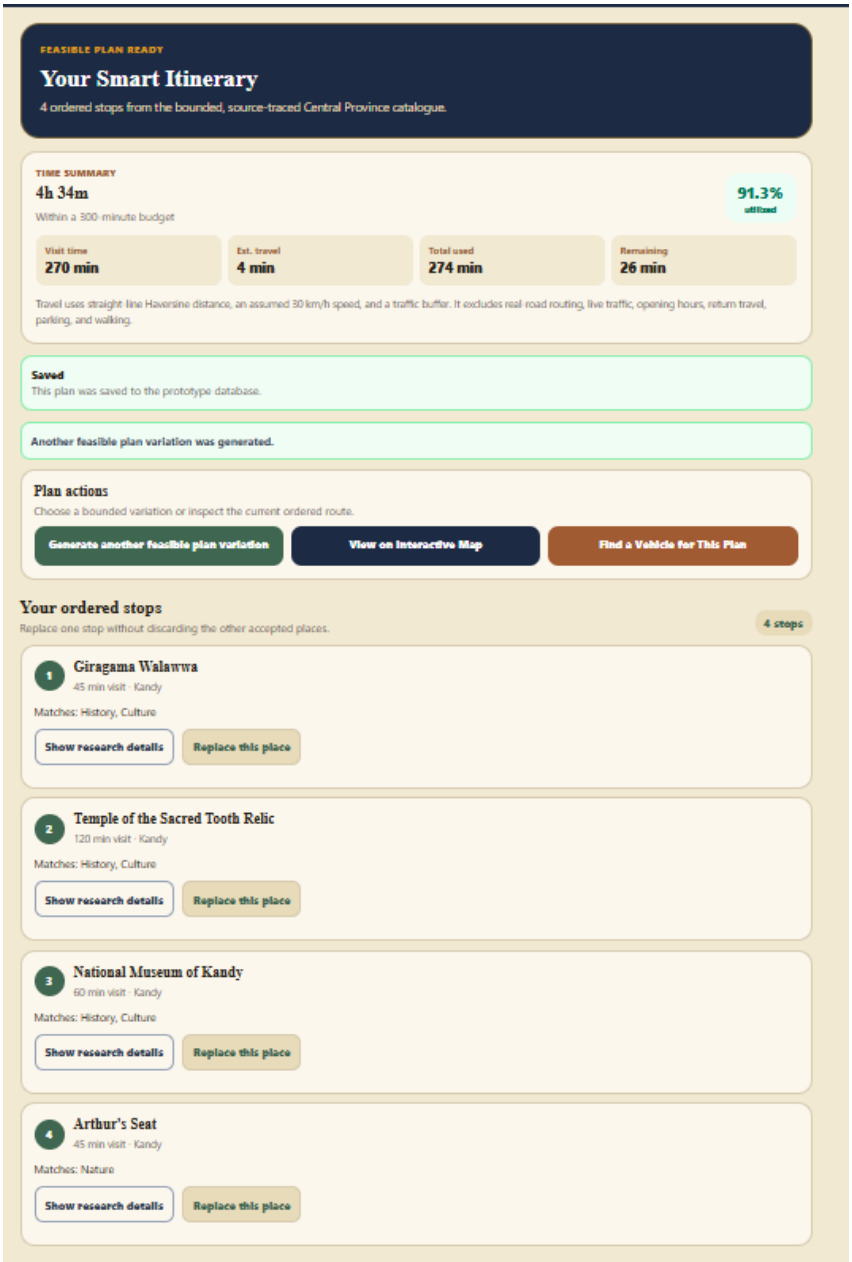


Figure 6.7. Successful feasible full-plan variation.

A separate successful variation returns a different four-stop plan ending at Arthur's Seat; the displayed total is 274 minutes with 91.3% utilisation.

6.3.8 Optional Gemini traveller guide

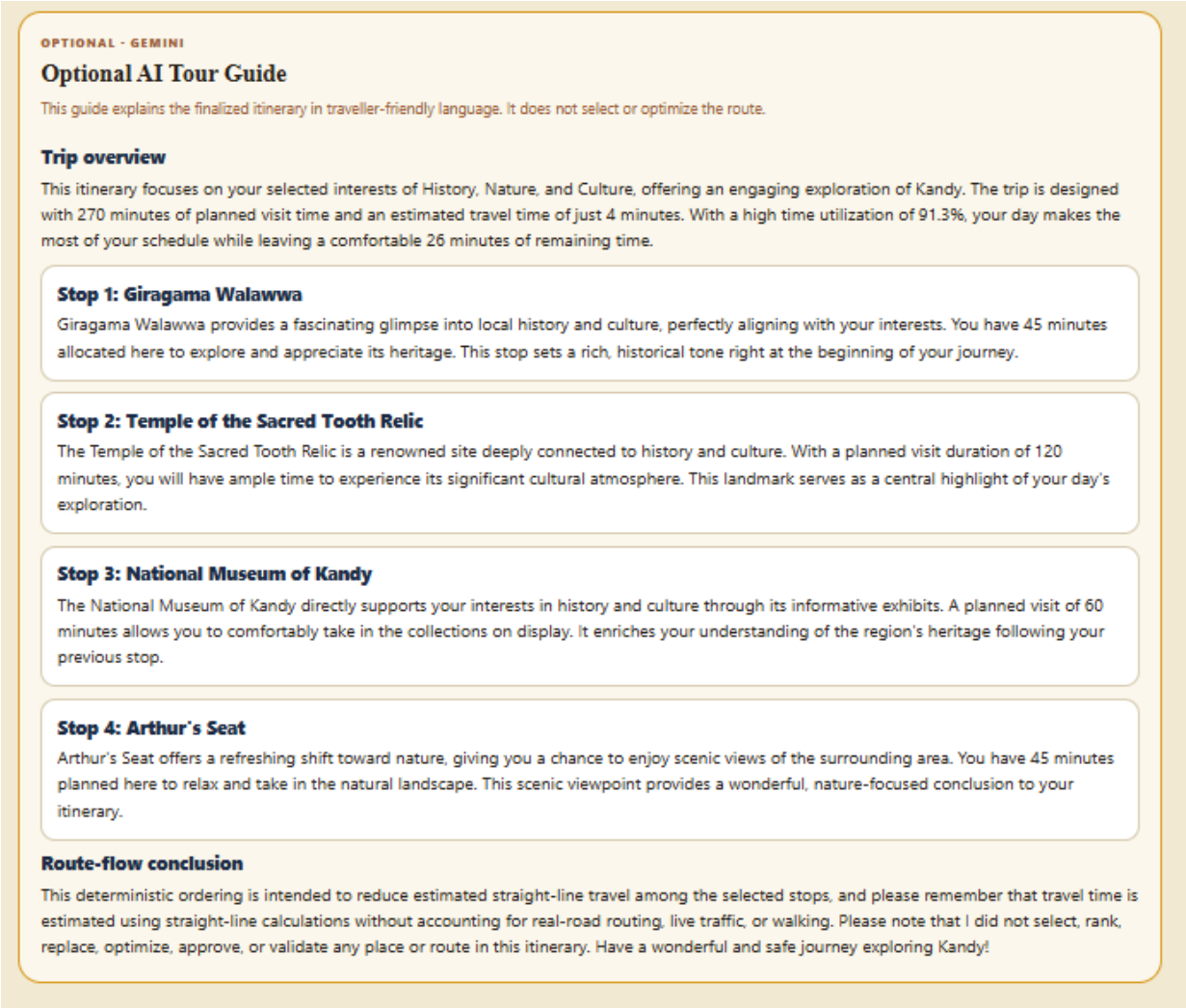


Figure 6.8. Optional Gemini traveller guide.

The optional Gemini guide describes the final four-stop variation in route order. The interface explicitly states that Gemini does not select or optimise the itinerary.

6.3.9 Relevance-gating and Genetic Algorithm selection stages

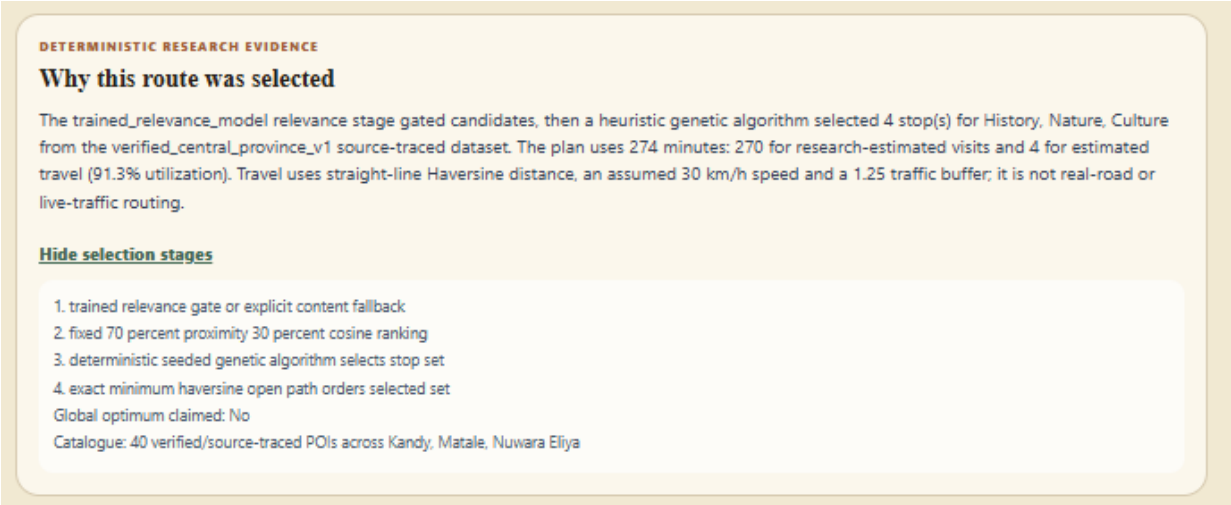


Figure 6.9. Relevance-gating and Genetic Algorithm selection stages.

The expanded selection stages identify relevance gating or explicit content fallback, fixed proximity prior, deterministic seeded heuristic GA, exact minimum-Haversine ordering and "Global optimum claimed: No".

The observed selection stages show a trained relevance gate or an explicit-content fallback, a proximity prior, heuristic GA-based candidate selection, and minimum-Haversine ordering. They do not show a globally optimality certificate or prove that every displayed plan used the trained classifier. This distinction matters when interpreting the human-heldout classification results alongside the runtime screenshots.

6.4 Summary of observed outcomes

The original screenshots establish that the signed-in prototype accepts the prescribed input and presents a feasible initial plan, research details, a rendered map, deterministic rationale, targeted replacement, full-plan variation and an optional guide. The evidence also includes actual saved-status feedback. It does not establish field reliability, nationwide location coverage or measured traveller satisfaction.

The visible selection-stage explanation expressly rules out claims of global optimality. Safety/vehicle functions remain outside the nine individual-component screenshots, avoiding an inappropriate transfer of a teammate's research outcomes.

7 DISCUSSION

7.1 Interpretation of research findings

The v1 classifier exceeded the selected baselines on the stated heldout set, but its margin is small and its balanced metrics constrain interpretation. The classifier narrows a candidate set; the GA and feasibility logic then construct an itinerary. Neither result justifies a claim that the system predicts the best possible tourist experience. The pre-synchronization demonstration shows a route can be replaced while improving time utilization, but it is not a usability study or independent effectiveness evaluation.

The model comparison demonstrates a bounded relevance-classification capability rather than universally strong personalisation. Its 68.33% accuracy exceeds the listed baselines slightly, while weak balanced and macro metrics reveal uneven class behaviour. These limitations make relevance gating a supporting technique, not a complete assessment of experience quality.

The screenshot sequence offers stronger evidence for the product workflow: generation and alteration of time-feasible itineraries with clearly displayed constraints. The observed replacement increases estimated time utilisation in one case, but no general causal improvement or user preference finding can be inferred from this single example.

7.2 Comparison with relevant techniques

The implementation reflects the tourist-trip-design distinction between interesting locations and feasible routes [1]. It uses classification concepts consistent with support-vector methods [3], heuristic optimisation with explicit feasibility constraints and a post-hoc explanation layer [4]. Unlike a road-routing product, this prototype uses Haversine-style geometry and explanatory travel assumptions rather than live road data.

The optional LLM guide demonstrates a separation of responsibilities: deterministic selection precedes language generation. This is relevant to the wider literature on large language models in recommender systems [6], but the screenshot cannot independently verify every generated sentence about the attractions.

7.3 Research and product contributions

The individual research contribution is an evaluated Linear SVM relevance gate paired with a bounded heuristic itinerary-construction process and transparent evidence limitations. The individual product contribution includes validated traveller requests, stop ordering, estimated time accounting, research-detail inspection, targeted replacement, unseen-plan variation and deterministic route explanations.

The team integration contribution is the provision of ordered itinerary output within an authenticated shared platform. Safety classification and vehicle recommendation remain separate ownership domains and are not used here as indicators of personal model effectiveness.

7.4 Limitations and threats to validity

The runtime catalogue has only 40 POIs and is limited to Central Province. The active model was evaluated on an earlier 20-POI Kandy set rather than independently across the current 40 POIs. The human-reviewed set is small, minority-class recall is weak, and no formal stakeholder study was completed. Haversine

estimates omit real-road distance, traffic, opening hours, parking and walking time. Gemini is optional and external. MongoDB is optional for generation but normal sign-in depends on it. The complete Python suite has four missing reviewer-workbook fixtures. Synchronized UI screenshots and the demonstration must be revalidated before submission.

Limitation	Future improvement
40-POI Central Province scope	Expand source-traced catalogue and validate per district
20-POI heldout evaluation	Conduct independent 40-POI and external validation
Haversine travel estimate	Use road-network and traffic-aware routing
No opening hours or walking model	Add time windows, parking and walk-time constraints
No formal user study	Conduct ethical stakeholder/usability evaluation
Missing reviewer workbooks	Recover approved separately supplied review artefacts

Table 7.1. Limitations and future improvements

The principal validity risks arise from the small heldout evaluation (60 examples), the earlier 20-POI evaluation scope, imbalanced relevance labels and the much larger current 40-POI runtime scope that was not independently re-evaluated. The 800 training/evaluation pairs should not be confused with unique tourist destinations.

The route estimate excludes real-road network distance, turn restrictions, congestion, parking, walking and opening-hour availability. The single Kandy demonstration and observed UI screenshots establish software behaviour but not general travel usefulness. The missing reviewer-workbook fixtures prevent an honest full-suite pass claim. An optional external guide introduces further narrative quality uncertainty.

7.5 Implications

As a bounded prototype, the component may help travellers inspect how a day plan fits selected interests and available hours. For operational deployment, the catalogue would need broader source coverage, validation across additional travellers and provinces, credible road-network estimates, and structured human evaluation. These are future-work requirements, not completed outcomes.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

The Smart AI Itinerary and Tour Plan Generator provides a bounded, reproducible itinerary workflow from validated traveller inputs to an ordered route, controlled regeneration and explanation. Its contribution is constrained and explicit: Linear SVM relevance gating, heuristic GA selection, Haversine-based estimated travel, deterministic explanation and optional post-hoc Gemini wording. Verification supports the runtime component through passing backend, frontend and focused Python tests, while the full suite limitation is reported transparently. Future work should extend the catalogue, evaluate relevance across the expanded scope, add road and time-window data, conduct user evaluation and capture genuine synchronized UI evidence.

8.2 Achievement of objectives

The component demonstrably accepts bounded input and returns an ordered itinerary from source-traced POIs. Recorded tests support core implementation behaviours, and the synchronized screenshots confirm multiple visible user actions. The model evaluation supports only moderate, scoped profile–POI relevance classification. Explicit caveats prevent mistaking the heuristic GA for a classifier or the travel estimate for accurate on-road navigation.

8.3 Individual contributions and future directions

The principal next steps are to validate relevance on the full 40-POI catalogue and additional regions, obtain a larger and more balanced human-labelled evaluation set, compare optimisation behaviour against documented baselines, add real road routing and time windows, measure real runtime latency, and conduct consent-based user evaluation. The report does not present these future studies as completed.

The final thesis therefore documents a functioning, evidence-bounded itinerary prototype rather than an unqualified nationwide or production-ready tourism recommender.

REFERENCES

[1] J. Ruiz-Meza and J. R. Montoya-Torres, "A systematic literature review for the tourist trip design problem: Extensions, solution techniques and future research lines," Operations Research Perspectives, vol. 9, Art. 100228, 2022, doi: 10.1016/j.orp.2022.100228.

[2] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou, "Mobile recommender systems in tourism," Journal of Network and Computer Applications, vol. 39, pp. 319-333, 2014, doi: 10.1016/j.jnca.2013.04.006.

[3] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, pp. 273-297, 1995, doi: 10.1007/BF00994018.

[4] Y. Zhang and X. Chen, "Explainable recommendation: A survey and new perspectives," Foundations and Trends in Information Retrieval, vol. 14, no. 1, pp. 1-101, 2020, doi: 10.1561/1500000066.

[5] C. F. F. Karney, "Algorithms for geodesics," Journal of Geodesy, vol. 87, no. 1, pp. 43-55, 2013, doi: 10.1007/s00190-012-0578-z.

[6] J. Lin et al., "How can recommender systems benefit from large language models: A survey," ACM Transactions on Information Systems, vol. 43, no. 2, Art. 28, pp. 1-47, 2025, doi: 10.1145/3678004.

[7] S. Li and K.-C. Choi, "Multi-objective optimization of itinerary planning based on weather changes and user preferences," Mathematics, vol. 14, no. 11, Art. 1949, 2026, doi: 10.3390/math14111949.

GLOSSARY

Term	Definition
Candidate gating	Filtering POIs before route optimization
Controlled exhaustion	A response that states no additional feasible alternative exists
Itinerary signature	Canonical POI-ID representation used to avoid cycling
Profile-POI pair	A row relating traveller-profile features to one POI
Runtime catalogue	The 40 POIs loaded by the service

Table G.1. Glossary

APPENDICES

Appendix A Reproducibility notes

The synchronized branch was sync/main-origin-integration-20261011 at merge commit ad691e42. Evidence recorded 65 passing backend tests, 18 passing frontend tests, passed lint, completed Expo export and 72 passing focused Python tests. The full Python suite ran 122 tests with four missing reviewer-workbook errors.

Appendix B Academic integrity, tool assistance and screenshot provenance

The figures in Chapter 6 were supplied as actual screenshots from the authenticated synchronized web application. They are labelled with chapter-based numbering and interpreted conservatively. The first view reflects the defined Kandy demonstration, replacement modifies one stop, and a subsequent full-plan variation produces the Arthur's Seat plan. No generated/mock screenshots or teammate Safety Analyzer outputs are used as personal research figures.

The local operational diagnosis involved correcting service endpoint configuration and enabling normal MongoDB-backed authentication. No secrets are printed in the thesis. Screenshots should be retained in original PNG form for independent review.

Report preparation and presentation involved assistance from ChatGPT for drafting, editing, and document formatting, and from Codex for local application startup, environment diagnosis, and verification support. The screenshots were manually captured from the genuine running application after normal authentication. This disclosure describes the assistance evidenced in the report-preparation workflow and does not assert an exhaustive record of every tool used during the research project.

The candidate remains responsible for verifying technical accuracy, source attribution, the university's current AI-use declaration requirements, and the final submitted document. The figure images document visible application states rather than independent end-user or field validation; no screenshot was fabricated or edited to depict a different software outcome.